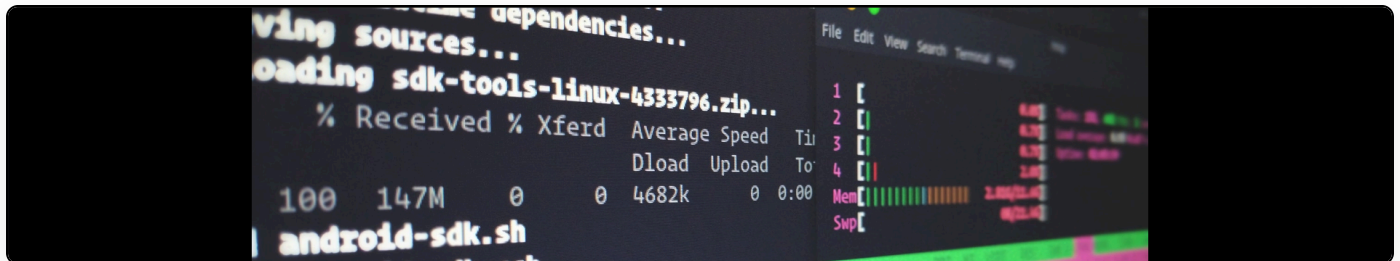


Right-To-Left and Left-To-Right characters

Cite as: Martin Paul Eve, "Right-To-Left and Left-To-Right characters", *eve.gd: Martin Paul Eve*, November 27, 2007, <https://doi.org/10.59348/hmhsb-xf838>.



There's been a fair bit of discussion going on at [slackers](#) on the security implications of the Unicode characters U+202D and U+202E which switch the left-to-right and right-to-left encoding of the following text.

So, what you appear to have in the source is:

```
<html id="test">
<head><title>A Test</title></head>
<body>
[REVERSE CHAR]<script>alert(1)</script>[UNREVERSE CHAR]
</body>
</html>
```

Which instantly leads to the question: is that text reversed and could therefore this be used for filter evasion?

To investigate, I created a simple c# program that creates 2 strings, the only difference between them being the inclusion of the reverse characters.

```
string s = "\r\n";
s += (char)int.Parse("202E", System.Globalization.NumberStyles.HexNumber);
s += TextBox1.Text;
s += (char)int.Parse("202D", System.Globalization.NumberStyles.HexNumber) + "\r\n";

string s2 = "\r\n";
s2 += TextBox1.Text;
s2 += "\r\n";
```

When cast to a char array, the output looked like this:

String containing evil characters: 13, 10, 8238, 60, 115, 99, 114, 105, 112, 116, 62, 97, 108, 101, 114, 116, 40, 49, 41, 60, 47, 115, 99, 114, 105, 112, 116, 62, 8237, 13, 10

String without: 13, 10, 60, 115, 99, 114, 105, 112, 116, 62, 97, 108, 101, 114, 116, 40, 49, 41, 60, 47, 115, 99, 114, 105, 112, 116, 62, 13, 10

I'll save you the hassle of looking and tell you now that, under .NET anyway, they are exactly the same. This means that any regex matching or `String.Contains()` functions will return the correct value and these representations will not evade filters. Whether PHP does the same, I shall leave for someone else to discover.

More disturbing however is the fact that these characters appear to be ignored by browser parsers meaning that putting one halfway through a word could lead to potential filter evasion as the string is not left in tact.